

CSCI-4208: Developing Advanced Web Applications

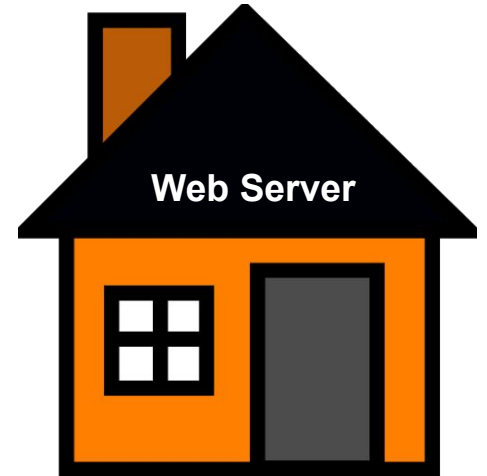
Week 8: Lecture 11
HTTP (via Web Client)

Overview: HTTP (Client Perspective)

1. HTTP - Basics
2. HTTP - URL
3. HTTP - Request/Response cycle
4. HTTP - Request message
5. HTTP - Response message
6. HTTP - Browser & HTML
7. HTTP - Browser & JS
8. HTTP - Alternative Web Clients

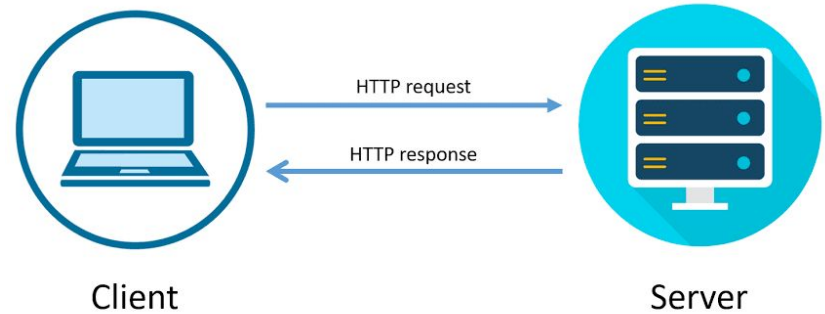
1. HTTP: Basics - Message Passing

The motivation behind HTTP is to provide a means for web clients and web servers to pass data between each other via messages.



1. HTTP: Basics - Properties

- **Hypertext Transfer Protocol (HTTP)** is an **application-layer protocol** for transmitting hypermedia documents, such as HTML.
- HTTP is designed for communication between: **web clients** and **web servers**.
- HTTP follows a **client-server model**, with a client opening a connection to make a request, then waiting until it receives a response from the server.
- HTTP is a **stateless** protocol, meaning that the server does not keep any data (state) between two requests.



1. HTTP: Basics - Application-Layer Protocol

In practice, there are many computers between a browser and the web server, such as: routers, modems, & more. Thanks to the layered design of the Web, these communications are hidden in the network and transport layers. HTTP is used at the application layer i.e. the browser, allowing HTML/JavaScript code an interface to the "outside".

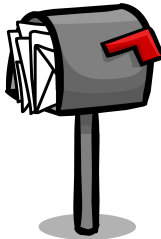


Postal Analogy:

We don't care how mail gets delivered.

Sender concerns are:

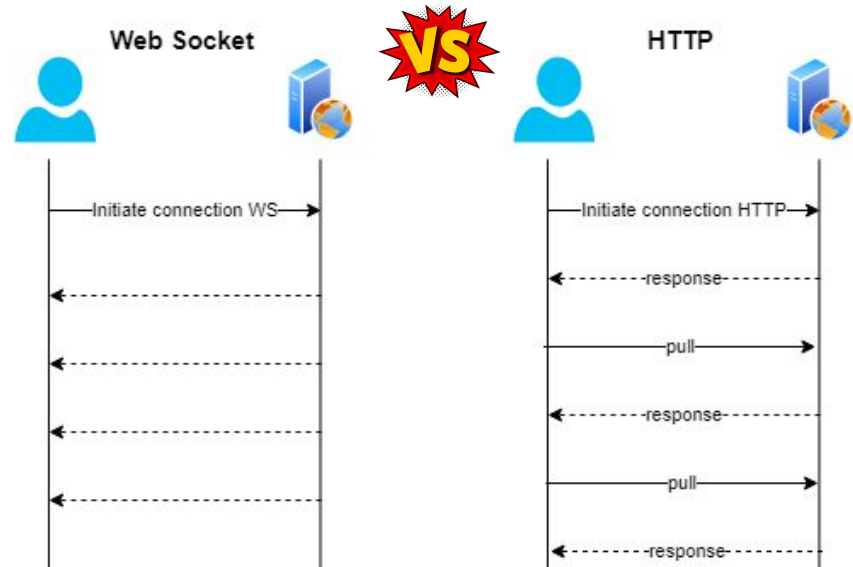
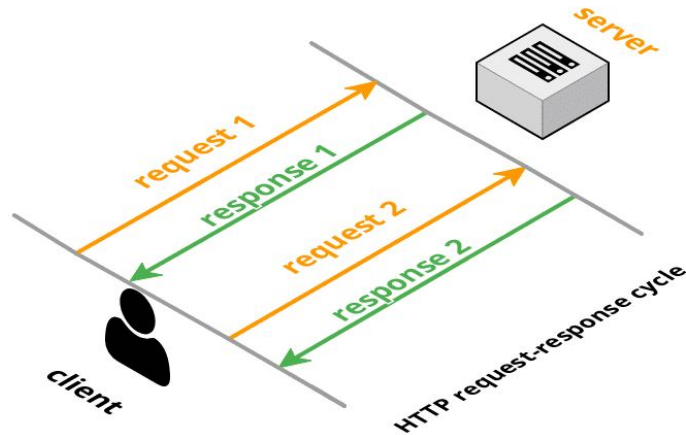
- *packaging*
- *addressing*
- *put in mailbox*



1. HTTP: Basics - Client-Server model

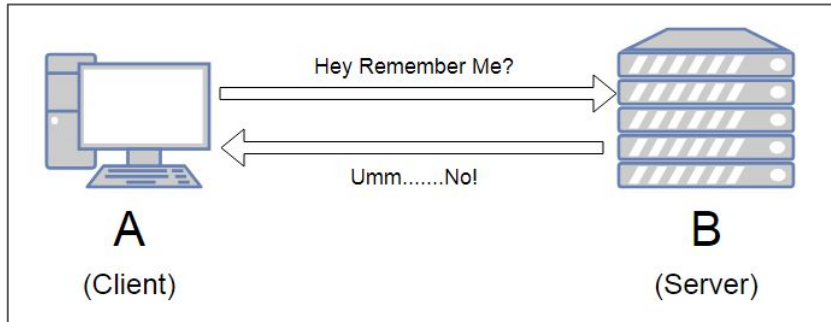
HTTP is **NOT** a continuous open connection. Once the response is delivered the connection is terminated. The connection must be initialized by the client with a request then a response given by the server.

Note: Browsers support alternate protocols between web servers & clients such as web sockets which establishes a two-way continuous stream of data.



1. HTTP: Basics - Stateless

Stateless means that the server does not store information about the client in between requests. Cookies are a way to keep HTTP stateless, but work around this restriction elegantly. The server does not need to store anything, and the client needs to send the cookies on every request in order to make the server know about certain aspects of the client. Cookie management is done on the client side by the browser.



Stateless HTTP



Stateless HTTP with Cookies

2. HTTP: URL

- Web Address
- Parameters
- Anchors
- JS URL API



2. HTTP: URL - Web Address

A URL (Uniform Resource Locator) is the address of a given unique resource on the Web. Each URL points to a unique resource. Such resources can be an HTML page, a CSS document, an image, etc.



Basics: anatomy of a URL

A URL is composed of different parts, some mandatory and others optional. The most important parts are highlighted on the URL above

2. HTTP: URL - Web Address

- **scheme:** the protocol that the browser must use to request the resource (*http, ws, ftp, smtp*)
- **domain:** The web server being requested. This is a domain name or an IP address.
- **port:** The "gate" used to access the resources on the web server. It's usually omitted if the web server uses the standard ports of the HTTP protocol (80 for HTTP and 443 for HTTPS) to grant access to its resources.
- **path:** The path to the resource on the Web server.

2. HTTP: URL - Parameters

- URLs may embed parameters provided to the Web server or Web client.
- Those parameters are a list of key/value pairs separated with the & symbol.
- The Web server or Web client can optionally use those parameters to do extra logic.

html?key1=value1&key2=value2#Some



2. HTTP: URL - Anchor

- URLs may embed an anchor to another part of the resource itself.
- An anchor represents a "bookmark" inside the resource, giving the browser the directions to show the content located at that "bookmarked" spot.
- The # is known as the fragment identifier.

value2#SomewhereInTheDocument



2. HTTP: URL - JS: URL API

- URL class
- SearchParams class
- Location Object Model
- encode/decode URI

2. HTTP: URL - JS: URL class

The built-in URL class provides an interface for creating & parsing URLs.

Creating a URL

The syntax to create a new URL object:

```
new URL(url, [base])
```

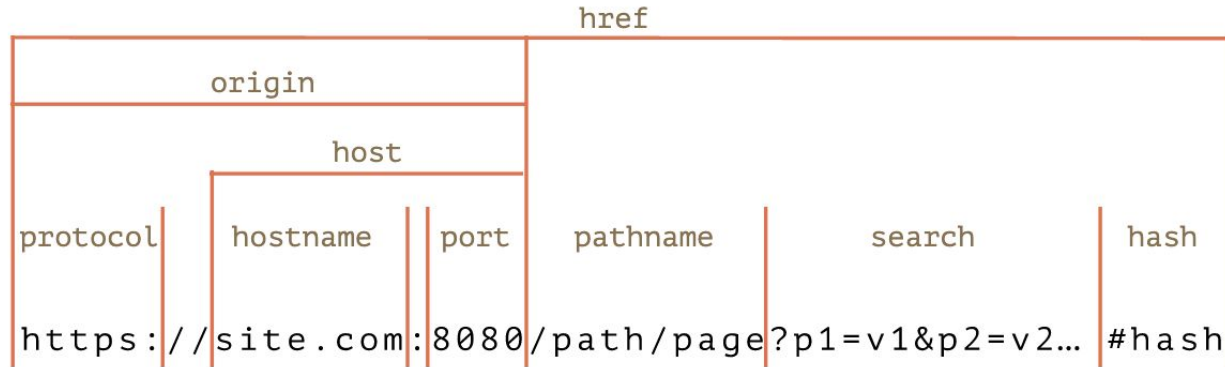
- **url** – the full URL or only path (if base is set, see below),
- **base** – an optional base URL: if set and **url** argument has only path, then the URL is generated relative to **base**.

Example

```
let url = new URL('https://javascript.info/profile/admin');  
let newUrl = new URL('tester', url);  
console.log(newUrl.href); // https://javascript.info/profile/tester
```

2. HTTP: URL - JS: URL class

URL Properties:



- `href` is the full url, same as `url.toString()`
- `protocol` ends with the colon character `:`
- `search` – a string of parameters, starts with the question mark `?`
- `hash` starts with the hash character `#`

2. HTTP: URL - JS: URL class

URL Properties - Demo

```
let url = new URL('http://example.org:8080/home/?key1=value1&key2=value2#idSelector');
console.log(url.href);
console.log(url.protocol);
console.log(url.hostname);
console.log(url.port);
console.log(url.host);
console.log(url.origin);
console.log(url.pathname);
console.log(url.search);
console.log(url.hash);
```


2. HTTP: URL - JS: URLSearchParams class

The built-in URLSearchParams class defines utility methods to work with the query string of a URL.

Creating a URLSearchParams

The syntax to create a new URLSearchParams object:

```
new URLSearchParams(queryString);
```

Example:

```
const query = "?key1=value1&key2=value2";  
const search = new URLSearchParams(query);  
const k1 = search.get("key1");  
const k2 = search.get("key2");  
console.log(k1,k2);
```

2. HTTP: URL - JS: URLSearchParams class

Methods:

- `append()` - Appends a specified key/value pair as a new search parameter.
- `delete()` - Deletes the given search parameter from the list of all search parameters.
- `get()` - Returns the first value associated with the given search parameter.
- `getAll()` - Returns all the values associated with a given search parameter.
- `has()` - Returns a Boolean indicating if such a given parameter exists.
- `set()` - Sets the value associated with a given search parameter to the given value.
- `sort()` - Sorts all key/value pairs, if any, by their keys.

2. HTTP: URL - JS: URLSearchParams class

URL Property: `searchParams`

The `searchParams` readonly property of the `URL` interface returns a `URLSearchParams` object allowing access to the GET decoded query arguments contained in the URL.

The syntax to create a new `URLSearchParams` object:

```
const urlSearchParams = url.searchParams
```

Example:

```
let url = new URL('http://example.org:8080/home/?key1=value1&key2=value2#idSelector');  
const search = url.searchParams;  
const k1 = search.get("key1");  
const k2 = search.get("key2");  
console.log(k1,k2);
```

2. HTTP: URL - JS: URLSearchParams class

Search Params: Pros/Cons

Query String embedded into a URL is an effective method for sending associative data to web servers/clients. However, it's not the only approach, as HTTP messages may include a body which contains this data. Here is a list of pros/cons for using search parameters to encode & send your data

Disadvantage:

- Only for public data, not a secure means of transmitting private data

Advantage:

- Human-readable, easy to declare & customize parameters
- Easy to test
- Serverless approach to send data into client, i.e. client-to-client data transfer
 - *Example: anchor tag from a source page to a target page with embedded search params*

2. HTTP: URL - JS: Location Object Model (LOM)

- The location object contains information about the current URL.
- The location object is part of the window object and is accessed through the window.location property.

Location Object Properties

Property	Description
<u>hash</u>	Sets or returns the anchor part (#) of a URL
<u>host</u>	Sets or returns the hostname and port number of a URL
<u>hostname</u>	Sets or returns the hostname of a URL
<u>href</u>	Sets or returns the entire URL
<u>origin</u>	Returns the protocol, hostname and port number of a URL
<u>pathname</u>	Sets or returns the path name of a URL
<u>port</u>	Sets or returns the port number of a URL
<u>protocol</u>	Sets or returns the protocol of a URL
<u>search</u>	Sets or returns the querystring part of a URL

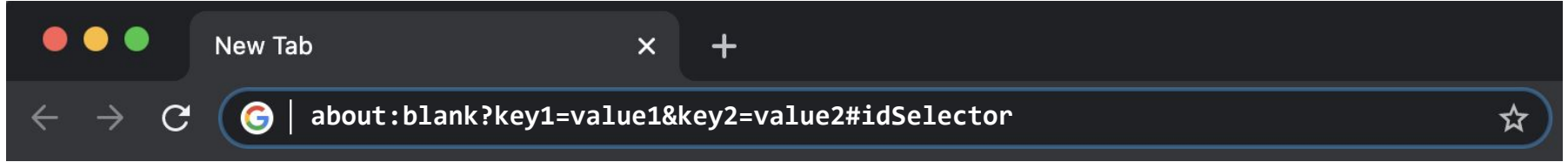
Location Object Methods

Method	Description
<u>assign()</u>	Loads a new document
<u>reload()</u>	Reloads the current document
<u>replace()</u>	Replaces the current document with a new one

2. HTTP: URL - JS: Location Object Model (LOM)

Example: (location object)

Step 1: Open URL in Address bar.



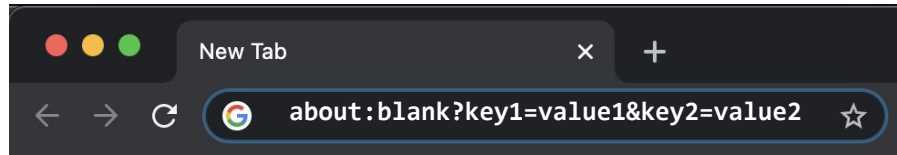
Step 2: Console log the location object and inspect its properties

```
console.log(location)
```

2. HTTP: URL - JS: Location Object Model (LOM)

Example: (Serverless approach for encoding data)

Step 1: Open URL with search params in Address bar. *(Suppose: href + search params encoded via DOM)*



Step 2: Access search params with the URL API

```
const queryString = location.search;
const search = new URLSearchParams(queryString);
const k1 = search.get('key1');
const k2 = search.get('key2');
console.log(k1,k2);
```

2. HTTP: URL - JS: encode/decode URI

URI

There's a standard [RFC3986](#) that defines which characters are allowed in a URI (uniform resource identifier) and which are not. Characters that are not allowed, must be encoded, for instance non-latin letters and spaces – are replaced with their UTF-8 codes, prefixed by %, such as %20

The URL API are built-in functions (global) for this purpose:

- `encodeURIComponent` – encodes URL as a whole.
- `decodeURIComponent` – decodes it back.
- `encodeURIComponent` – encodes a URL component, such as a search parameter, or a hash, or a pathname.
- `decodeURIComponent` – decodes it back.

2. HTTP: URL - JS: encode/decode URI

encodeURIComponent

Encodes a string into a properly formatted URI as a whole

Example:

```
const text = "This string contains spaces - not valid as a URI";  
const uri = encodeURIComponent(text);  
console.log(uri);
```

decodeURI

Decodes a URI into a string

Example:

```
const uri = "This%20string%20contains%20spaces%20-%20not%20valid%20as%20a%20URI";  
const text = decodeURI(uri);  
console.log(text);
```

2. HTTP: URL - JS: encode/decode URI

encodeURIComponent

encodes a URL component, such as a search parameter, or a hash, or a pathname.

encodeURIComponent is same as encodeURIComponent, but in addition, also handles characters: #, \$, &, +, ,, /, :, ,, =, ? and @

Example:

```
const text = "#$&+/:;=?@";  
const uri = encodeURIComponent(text);  
console.log(uri);
```

decodeURIComponent

Decodes a URI component into a string

Example:

```
const uri = "%23%24%26%2B%2F%3A%3B%3D%3F%40";  
const text = decodeURIComponent(uri);  
console.log(text);
```

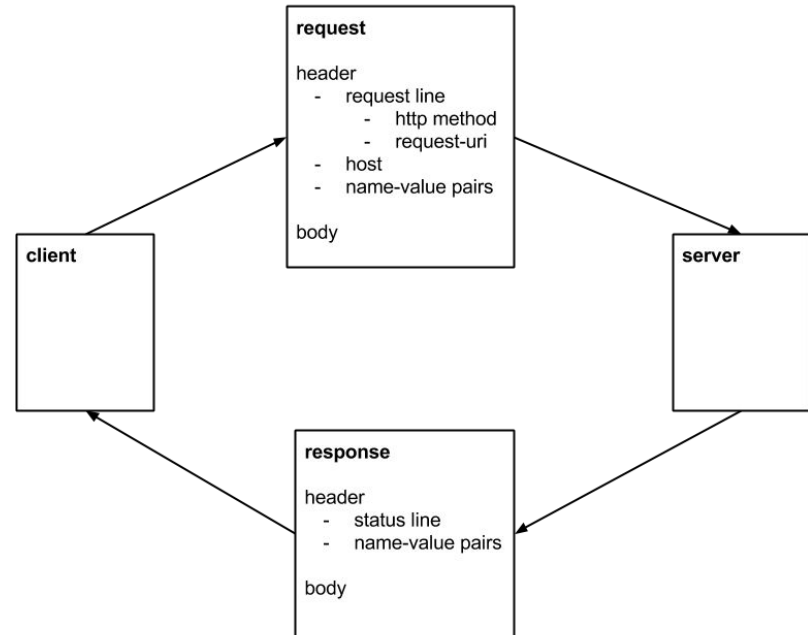
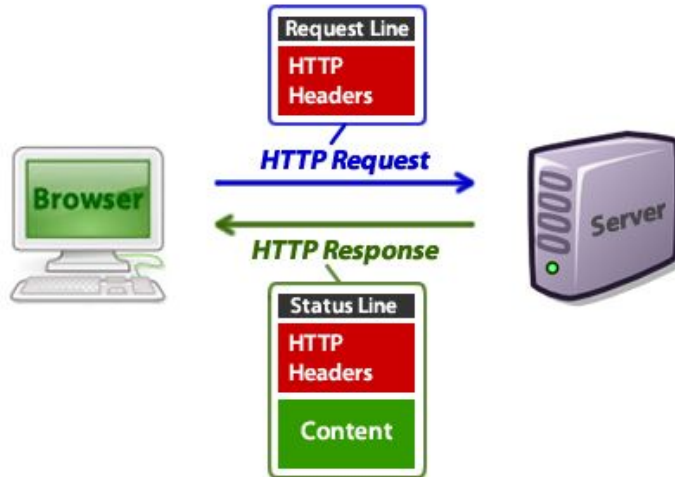
2. HTTP: URL - JS: encode/decode URI

Example (encode String for searchParams):

```
let music = encodeURIComponent('Rock&Roll');  
let url = `https://google.com/search?q=${music}`;  
console.log(url); // https://google.com/search?q=Rock%26Roll
```

3. HTTP: Request/Response cycle

Request/Response cycle



3. HTTP: Request/Response cycle

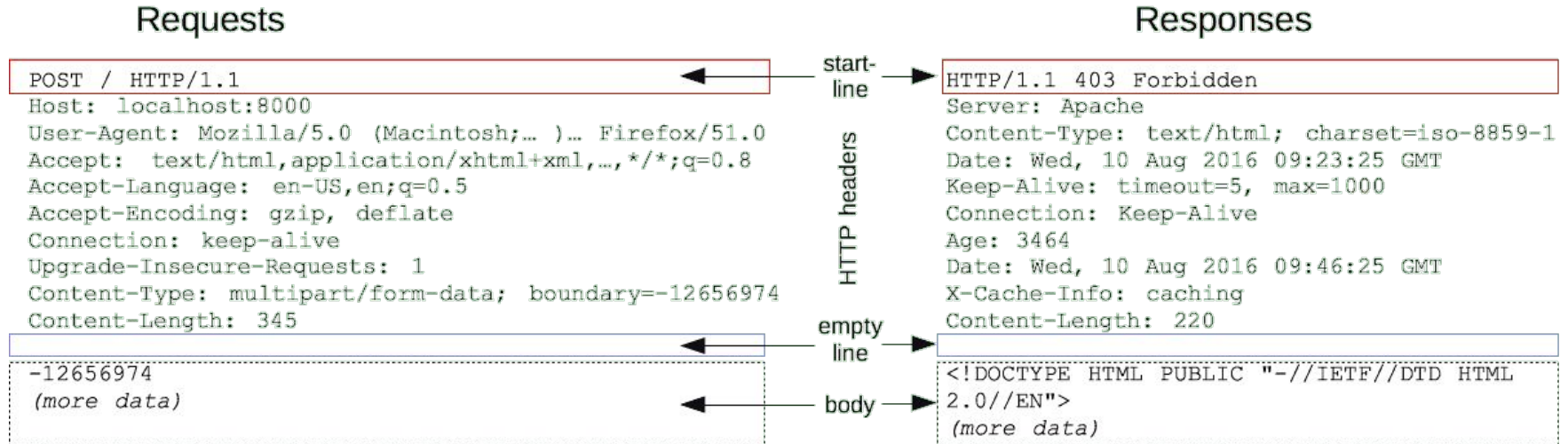
- Clients and servers communicate by exchanging individual messages (as opposed to a stream of data).
- The messages sent by the client, usually a Web browser, are called requests
- The messages sent by the server as an answer are called responses.

3. HTTP Request/Response cycle

- A web browser is an HTTP client, sending requests to server machines. When the browser user enters file requests by either "opening" a web file (typing in a URL) or clicking on a hypertext link, the browser builds an HTTP request and sends it to the Internet Protocol address (IP address) indicated by the URL.
- A web server machine contains an HTTP daemon, a program that is designed to wait for HTTP requests and handle them when they arrive. The HTTP daemon receives the request and sends back the requested file or files associated with the request.

3. HTTP: Request/Response cycle

HTTP messages are how data is exchanged between a server and a client. There are two types of messages: *requests* sent by the client to trigger an action on the server, and *responses*, the answer from the server. These messages are human-readable.



4. HTTP: Request message

- HTTP Verbs
- Structure of Request Message
- Request message: Start Line
- Request message: Header
- Request message: Body

4. HTTP: Request message - Verbs

HTTP Verbs

Similar to how spoken language has different kinds of statements(Interrogative, Informative, Imperative, etc.), there are several types of HTTP methods of clients requests with specific roles. Some of the most important methods are:

- GET
- POST
- PUT
- DELETE

4. HTTP: Request message - Verbs

GET

GET methods are the HTTP requests which a client makes to a server for getting resources from the server.
For e.g., whenever you open Google in your browser, it makes a GET request to google's server.

POST

POST methods are the HTTP requests which a client makes for sending new data to a server.
For e.g. when you add a tweet, your application makes a POST request to twitter's server.

PUT

PUT methods are almost like POST methods, instead of adding information to a server, they are intended to update existing information on the server.
For e.g. when you change your profile pic on Twitter, your application makes a PUT request to the server.

DELETE

DELETE methods are the HTTP requests which a client makes to delete the existing data from the server.
For e.g. whenever you delete a tweet on Twitter, your application makes a DELETE request to the server.

4. HTTP: Request message - Verbs - considerations

Safety

An HTTP method is said to be safe if it does not change the state of a resource when invoked.

- GET and HEAD methods are safe, they retrieve data and no matter how many times they are called the state of the resource is not changed.
- POST is sometimes safe if it is used to retrieve data and does not create or update. Some purists may say that POST should only be used to create a resource, but in many real-world implementations you will see POST used for complex queries or data retrieval.

4. HTTP: Request message - Verbs - considerations

Idempotent

An HTTP method is idempotent if it can be called multiple times and the end result is the same.

- GET, HEAD, PUT and DELETE methods are idempotent; they can be called any number of times with the same result.
- Some will argue that DELETE is not idempotent because the second, third, etc. calls can result in an error message, but there are no side-effects to the resource so it is idempotent.

4. HTTP: Request message - Structure

Request Message - Structuring a Request (Technical explanation)

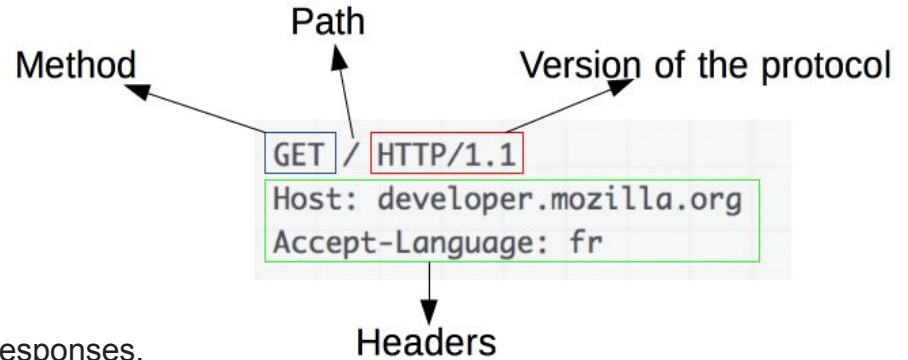
Once the connection is established to the web server, the user-agent can send the request (a user-agent is typically a web browser). A client request consists of text directives, separated by CRLF (carriage return, followed by line feed), divided into three blocks:

1. **Start line:** The first line contains a request method followed by its parameters:
 - the path of the document, *i.e. an absolute URL without the protocol or domain name*
 - the HTTP protocol version
2. **Header:** Subsequent lines represent an HTTP header, giving the server information about what type of data is appropriate (e.g., what language, what MIME types), or other data altering its behavior. These HTTP headers form a block which ends with an empty line.
3. **Body:** The final data block is optional, which may contain further data mainly used by the POST method.

4. HTTP: Request message - Structure

Requests consists of the following elements:

- **(Required) Start line**
 - An HTTP method, (i.e. verb)
 - The path of the resource to fetch;
 - The version of the HTTP protocol.
- **(Optional) Headers:**
 - convey additional information for the servers.
- **(Optional) Body:**
 - for some methods like POST, similar to those in responses, which contain the resource sent.



4. HTTP: Request message - Start Line

Request's *start-line* contain three elements:

1. *HTTP method*: a verb or a noun, that describes the action to be performed.
2. *Request target*, a URL, or the absolute path of the protocol, port, and domain are usually characterized by the request context. The format of this request target varies between different HTTP methods. The most common format is as an absolute path followed by a '?' and query string.
3. *HTTP version*, acts as an indicator of the expected version to use for the response.

4. HTTP: Request message - Headers

HTTP headers let the client and the server pass additional information with an HTTP request or response. An HTTP header consists of its case-insensitive name followed by a colon (:), then by its value. Whitespace before the value is ignored.

Header types for a Request message:

- **General headers**, Apply to the message as a whole.
- **Request headers**, Modify the request by specifying it further.
- **Representation metadata headers**: Describe the encoding and format of the message body.

```
POST / HTTP/1.1
Host: localhost:8000
User-Agent: Mozilla/5.0 (Macintosh;... )... Firefox/51.0
Accept: text/html,application/xhtml+xml,...,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Connection: keep-alive
Upgrade-Insecure-Requests: 1
Content-Type: multipart/form-data; boundary=-12656974
Content-Length: 345

-12656974
(more data)
```

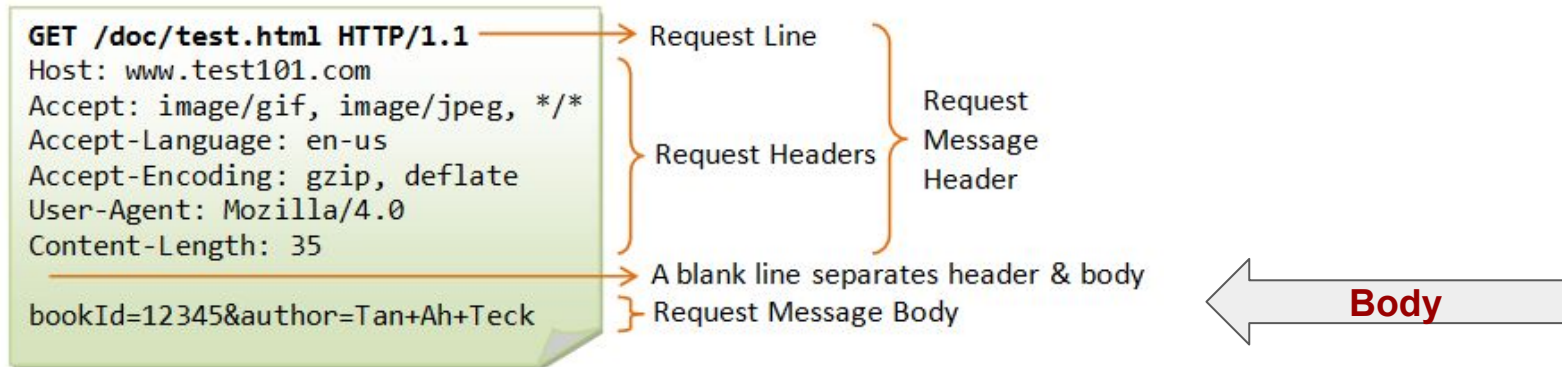
The diagram illustrates the categorization of HTTP request headers. It shows a list of headers from an HTTP POST request. Three colored arrows point to specific groups of headers: a red arrow points to 'Request headers' (Accept, Accept-Language, Accept-Encoding), a green arrow points to 'General headers' (Connection, Upgrade-Insecure-Requests), and a blue arrow points to 'Representation headers' (Content-Type, Content-Length). The headers are also visually grouped by background color: red for request headers, green for general headers, and blue for representation headers.

4. HTTP: Request message - Body

Most request types don't require a body, like GET or DELETE. But some requests send data to the server in order to update it: as often the case with POST requests (containing HTML form data).

Body for a Request message:

- **Single-resource bodies**, consists of a single file, defined by two headers: Content-Type & Content-Length.
- **Multiple-resource bodies**, consists of a multipart body, each containing a different bit of information. This is typically associated with HTML Forms.



4. HTTP: Request message - Body

Body: Pros/Cons

Using the HTTP Response Body is a secure means to send data from a HTTP client to a HTTP server as the data is encapsulated into the message. However, it's not the only approach, as URLs may embed parameters.

Here is a list of pros/cons for using the HTTP Response body to send your data

Disadvantage:

- More restrictive to send, must use an HTML form or JavaScript fetch method.
- Not as easy to test, cannot just send request from the browser's URL bar
- Requires a server to receive data, i.e. does **not** allow for client-to-client data transfer
 - *Example: anchor tag from a source page to a target page with embedded search params*

Advantage:

- Secures & Hides data, suitable for sending private or public data
- No limit to length of data

5. HTTP: Response message

- HTTP Status Codes
- Structure of Response Message
- Request message: Status Line
- Request message: Header
- Request message: Body

5. HTTP: Response message - Status Codes

HTTP status codes

HTTP Status codes are part of the HTTP Response. It helps the client understand what happened to the request. It's a 3 digit numeric code (like 404, 502), which has an appropriate definition in HTTP spec, a reason phrase is also returned along with it. For eg. 404: Page not found!

There are 5 types of HTTP status codes: *(based on the starting digit)*

- 1xx: Information Responses
- 2xx: Successful Responses
- 3xx: Redirect Responses
- 4xx: Client Errors
- 5xx: Server Errors

5. HTTP: Response message - Status Codes

CODE	Constant	Description
200	OK	Successfully executed
201	CREATED	Successfully executed and a new resources has been created. The response body is either empty or contains a URI(s) of the created resource. The location header should should also contain the new URI
202	ACCEPTED	The request was valid and has been accepted but has not yet been processed. The response should include a URI to poll for status updates on the request. Allows for asynchronous request
204	NO_CONTENT	The request was successful but the server did not have a response
301	MOVED_PERMANENTLY	The new location should be returned in the response
400	BAD_REQUEST	Malformed or invalid request
401	UNAUTHORIZED	Invalid authorization credentials
403	FORBIDDEN	Disallowed request. Security error
404	NOT_FOUND	Resource not found
405	METHOD_NOT_ALLOWED	Method (verb) not allowed for requested resource. Response will provide an "Allow" header to indicate what is allowed
415	UNSUPPORTED_MEDIA_TYPE	Client submitted a media type that is incompatible for the specified resource.
500	INTERNAL_SERVER_ERROR	Catchall for server processing problem
503	SERVICE_UNAVAILABLE	Response in the face of too many request

5. HTTP: Response message - Structure

Response Message - Structuring a Response (Technical explanation)

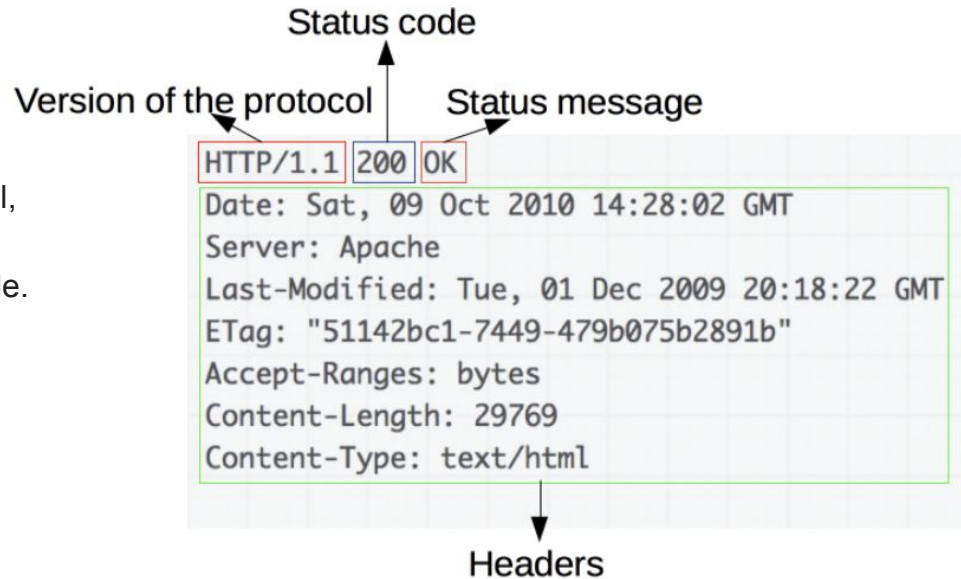
After the connected agent has sent its request, the web server processes it, and ultimately returns a response. Similar to a client request, a server response is formed of text directives, separated by CRLF, divided into three blocks:

1. **Status line**, The first line consists of an acknowledgment of the HTTP version used, followed by a status request (and its brief meaning in human-readable text).
2. **HTTP headers**, Subsequent lines represent specifics, giving the client information about the data sent (e.g. type, data size, compression algorithm used, hints about caching). HTTP headers form a block ending with an empty line.
3. **HTTP body**: The final block is a data block, which contains the optional data.

5. HTTP: Response message - Structure

Response consists of the following:

- **(Required) Start line:**
 - Version of HTTP protocol.
 - [Status code](#), indicates if request was successful, or not, and why.
 - Status message, a description of the status code.
- **(Optional) Headers**
 - convey additional information for the servers.
- **(Optional) Body**
 - The body contains the fetched resource.



5. HTTP: Response message - Status line

The start line of an HTTP response, called the *status line*, contains the following information:

1. The *protocol version*, usually HTTP/1.1.
2. A *status code*, indicating success or failure of the request. Common status codes are [200](#), [404](#), or [302](#)
3. A *status text*. A brief, purely informational, textual description of the status code to help a human understand the HTTP message.

A typical status line looks like: HTTP/1.1 404 Not Found.

5. HTTP: Response message - Headers

[HTTP headers](#) for responses structured with a case-insensitive string followed by a colon (':') and a value.

Header types for a Request message:

- **General headers**, Apply to the message as a whole.
- **Request headers**, Modify the request by specifying it further.
- **Representation headers**: Describe the encoding and format of the message body.

```
HTTP/1.1 200 OK
Access-Control-Allow-Origin: *
Connection: Keep-Alive
Content-Encoding: gzip
Content-Type: text/html; charset=utf-8
Date: Wed, 10 Aug 2016 13:17:18 GMT
Etag: "d9b3b803e9a0dc6f22e2f20a3e90f69c41f6b71b"
Keep-Alive: timeout=5, max=999
Last-Modified: Wed, 10 Aug 2016 05:38:31 GMT
Server: Apache
Set-Cookie: csrftoken=.....
Transfer-Encoding: chunked
Vary: Cookie, Accept-Encoding
X-Frame-Options: DENY
```

Response headers

Representation headers

General headers

(body)

5. HTTP: Response message - Body

The last part of a response is the body. Not all responses have one: responses with a status code that sufficiently answers the request without the need for corresponding payload (like [201 Created](#) or [204 No Content](#)) usually don't.

Body for a Response message:

- Single-resource bodies, consists of a single file of known length,
 - defined by the two headers: [Content-Type](#) and [Content-Length](#).
- Single-resource bodies, consists of a single file of unknown length,
 - encoded by chunks with [Transfer-Encoding](#) set to chunked.
- [Multiple-resource bodies](#), consists of a multipart body, each containing a different section of information.

6. HTTP: Browser/HTML - HTML API

Browsers send HTTP request messages. These request may be initiated via HTML elements.

This section will examine the following

- Browser
- `<a>`, `<iframe>`: HTML Anchors/Iframes
- `<form>`, `<input>`: HTML Forms

6. HTTP: Browser/HTML - Browser

The browser's address bar triggers a GET request to the given URL and handles the response message.

When an HTML page loads into the browser, any element with the **src** attribute causes the browser to issue a GET request to the given URL.

6. HTTP: Browser/HTML - <a>, <iframe>

Anchor & Inline Frames may trigger GET requests. Anchor hyperlink other media into the HTML document. Inline Frame is a window inside a browser window. Iframes may be the target to both anchors and form submissions.

- Anchors <a>
 - **href:** browser sends a GET request to the URL when clicked.
 - **target:** the browser window to handle the response
- Inline frames <iframe> </iframe>
 - **src:** issues a GET request when the document loads

6. HTTP: Browser/HTML - <form>, <input>

Forms may send GET or POST requests. Forms may contain input elements that send the values to the web server. A submit event on the form sends it via its action and method.

- **form** <form></form>
 - **method:** The HTTP method used to submit the form: GET or POST
 - **action:** The URL that processes the form submission.
 - **target:** Indicates where to display the response after submitting the form.
 - **onsubmit:** Execute a JavaScript when a form is submitted
- **inputs** <input> </input>
 - **name:** the key name to be used by the web server to label this value
 - **value:** the value of this input to associate with the key:value pair
 - **id:** the identifier used to by client to access element in javascript interpreter

6. HTTP: Browser/HTML - <form>, <input>

Demo (W3 School - server)

```
<form action="https://w3schools.com/action_page.php">  
  <label for="fname">First name:</label><br>  
  <input type="text" id="fname" name="fname" value="John"><br>  
  <label for="lname">Last name:</label><br>  
  <input type="text" id="lname" name="lname" value="Doe"><br><br>  
  <input type="submit" value="Submit">  
</form>
```

Demo (Google - server)

```
<form action="http://www.google.com/search" method="get">  
  <label for="q">Topic:</label>  
  <input type="text" id="topic" name="q"><br><br>  
  <input type="submit" value="Submit">  
</form>
```

6. HTTP: Browser/HTML - <form>, <input>

Pros/Cons HTML forms

HTML forms provide a built-in approach to send HTTP request messages to a web server. However, there exists other approaches such as the Javascript Fetch API.

Disadvantage:

GET & POST methods only. Cannot use DELETE, PUT methods, even though they are HTTP standard.

Advantage:

Easy to use without requiring JavaScript.

7. HTTP: Browser/JS - Fetch API

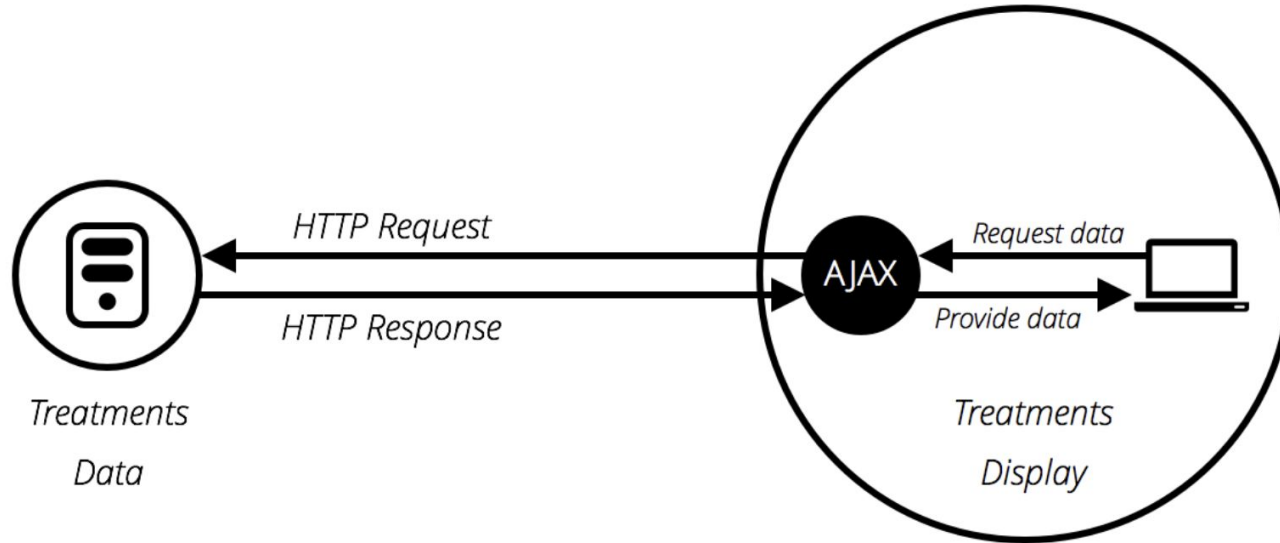
JavaScript can send network requests to the server and load new information whenever it's needed. "AJAX" (abbreviated Asynchronous JavaScript And XML) for network requests from JavaScript.

Fetch API:

- fetch method (global)
- Request class
- Response class
- Header class
- Body class

7. HTTP: Browser/JS - Fetch API

The Ajax model involves using a web API as a proxy to more intelligently request data rather than just having the browser reload the entire page.



7. HTTP: Browser/JS - Fetch API: fetch function

The builtin-in method `fetch()` is used to send a network request and get information from the server.

The basic syntax is:

```
let promise = await fetch(url, [options])
```

- **url** – the URL to access. Either as a String or Request object (*the Request option may also contain its options*)
- **options** – optional parameters: method, headers etc. Without options, this is a simple GET request.

This is an asynchronous function, whereby the client must wait for the server to send back a response. Thus we must use the **async/await** keywords whenever calling this function.

7. HTTP: Browser/JS - Fetch API: fetch function

Demo

- See Lab for examples with Fetch API
- Demo at end of this slide section

7. HTTP: Browser/JS - Fetch API: Request class

Request class

The Request class represents a resource request. Create a new Request object using its constructor.

The basic syntax is:

```
var myRequest = new Request(input[, init]);
```

- **input:** A string containing the direct URL of the resource you want to fetch.
- **init:** An options object containing any custom settings to apply to the request. The possible options are:
 - **method:** The request method, e.g., GET, POST. The default is GET.
 - **headers:** Any headers you want to add to your request, contained within a [Headers](#) object.
 - **body:** Any body that you want to add to your request: this can be a [Blob](#), [BufferSource](#), [FormData](#), [URLSearchParams](#), [USVString](#), or [ReadableStream](#) object.
 - **redirect:** The redirect mode to use: `follow`, `error`, or `manual`. The default is `follow`.

7. HTTP: Browser/JS - Fetch API: Request class

Example:

```
const options = {method: 'POST', body: '{"foo": "bar"}'};  
const request = new Request('https://example.com', options);  
  
console.log(request.url);  
console.log(request.method);  
console.log(request.credentials);  
console.log(request.bodyUsed);
```

7. HTTP: Browser/JS - Fetch API: Response class

Response class

The Response class represents the response to a request.

The properties of a response are:

- `status` – HTTP status code, e.g. 200.
- `ok` – boolean, `true` if the HTTP status code is 200-299.

The methods, which are promise-based (`async/await`):

- `response.text()` – read the response and return as text,
- `response.json()` – parse the response as JSON,
- `response.formData()` – return the response as `FormData` object (explained in the [next chapter](#)),
- `response.blob()` – return the response as `Blob` (binary data with type),
- `response.arrayBuffer()` – return the response as `ArrayBuffer` (low-level representation of binary data),

7. HTTP: Browser/JS - Fetch API: Header class

The Headers class of the Fetch API allows you to perform various actions on HTTP request and response headers.

These actions include retrieving, setting, adding to, and removing headers from the list of the request's headers.

A Headers object has an associated header list, which is initially empty and consists of zero or more name and value pairs. You can add to this using methods like `append()`

more info:

<https://developer.mozilla.org/en-US/docs/Web/API/Headers>

7. HTTP: Browser/JS - Fetch API: Body class

The Body class of the Fetch API represents the body of the response/request, allowing you to declare what its content type is and how it should be handled.

Body is implemented by both Request and Response.

This provides these objects with an associated body (a stream), a used flag (initially unset), and a MIME type (initially the empty byte sequence).

more info:

<https://developer.mozilla.org/en-US/docs/Web/API/Body>

7. HTTP: Browser/JS - Fetch API: fetch method

Demo:

Step 1: From terminal: launch http-server (using node)

```
> http-server
```

Step 2: from browser: send fetch request

```
await fetch('http://localhost:8080',{method:'post'})  
await fetch('http://localhost:8080',{method:'get'})  
await fetch('http://localhost:8080',{method:'put'})  
await fetch('http://localhost:8080',{method:'delete'})
```

7. HTTP: Browser/JS - Fetch API - Summary

Pros/Cons Fetch API

Javascript Fetch API provides a powerful interface to customize and send HTTP request and manage HTTP responses.

Disadvantage:

- Requires Async JavaScript

Advantage:

- Highly customizable and configurable
- Only way to access to all standard HTTP methods: GET, POST, PUT, DELETE
- Manage both HTTP requests & HTTP responses
- Capable of executing JS code in response to a HTTP response.

7. HTTP: Browser/JS - Axios

An alternative to the Fetch API is Axios, a JavaScript library that offers comprehensive HTTP network requests/response.

More info:

<https://github.com/axios/axios>

8. HTTP - Alternative Web Clients

How to make an HTTP request?

Instead of your browser, there are several other ways to make HTTP requests and accept responses from servers, these are incredibly useful for developers to test their applications before deploying. We'll look into 2 most popular way of making HTTP requests:

- Using CLI
- Using GUI

Making HTTP requests with CLI

cURL is like a web-browser, but for the command line.

Making HTTP requests with GUI

Postman is one of the most famous apps for it.

HopScotch is a web app used for sending HTTP request (<https://hoppscotch.io/>)

The End.

COMING SOON... TO A MOODLE NEAR YOU!

Labs

Portfolio Contact form, HiLo Scoreboard

Homework

Browser App using a REST API